



CI/CD Security, Pentesting y OWASP Top 10

Taller de Seguridad Web – OWASP Top 10

Carlos Rodríguez Contreras · GitHub: [karrocon](#)

Agenda del día

Sesión 1 (2h)

Bloque	Tema
Repaso	Warm-up día 7
T19	CI/CD Security + DevSecOps
Lab	GitHub Actions workflow
T20	Pentesting con OWASP ZAP
Lab	Escaneo ZAP en Docker

Sesión 2 (2h)

Bloque	Tema
T20+	Informe de auditoría
T21	OWASP Top 10 – mapa completo
Lab	Mapear el curso a OWASP Top 10
Cierre	Recursos + certificaciones

Objetivos del día

Al terminar hoy serás capaz de:

1.  Crear un pipeline CI/CD con seguridad integrada (shift-left)
2.  Ejecutar OWASP ZAP para descubrir vulnerabilidades automáticamente
3.  Estructurar un informe de auditoría profesional con CVSS
4.  Mapear TODO el curso a las 10 categorías del OWASP Top 10

💡 **Último día.** Hoy unimos TODAS las piezas y os damos el roadmap para seguir aprendiendo.

T19: CI/CD Security

DevSecOps — Seguridad en cada push

¿Qué es DevSecOps?

Tradicional: Dev → QA → Release → Pentest (demasiado tarde)
DevSecOps: Dev+Sec → QA+Sec → Release+Sec (seguridad en cada paso)

Shift-Left: Mover la seguridad lo más a la IZQUIERDA posible:

Fase	Herramienta	Qué detecta
Pre-commit	git-secrets, husky	Secretos hardcodeados
Build	npm audit, Snyk	Dependencias con CVEs
Test	SAST (CodeQL)	Patrones inseguros en código
Deploy	DAST (ZAP)	Vulnerabilidades en la app desplegada
Runtime	WAF + monitoring	Ataques en tiempo real

SAST vs DAST vs SCA

Tipo	Qué analiza	Cuándo	Ejemplos
SAST	Código fuente (sin ejecutar)	Build	CodeQL, Semgrep, SonarQube
DAST	App ejecutándose (black-box)	Post-deploy	OWASP ZAP, Burp Suite
SCA	Dependencias (CVEs conocidos)	Build	npm audit, Snyk, Dependabot

SAST → "Tu código tiene un patrón de SQLi en línea 42"

DAST → "La URL /api/products?q=' devuelve error SQL"

SCA → "jsonwebtoken@8.5.1 tiene CVE-2022-xxxxx"

GitHub Actions – Pipeline de seguridad

```
name: Security Pipeline
on: [push, pull_request]

jobs:
  security:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Install dependencies
        run: npm ci

      # SCA – Dependencias vulnerables
      - name: Audit dependencies
        run: npm audit --audit-level=high

      # Secretos filtrados
      - name: Scan for secrets
        uses: gitleaks/gitleaks-action@v2

      # SAST – Análisis de código
      - name: CodeQL Analysis
        uses: github/codeql-action/analyze@v3
        with:
          languages: javascript-typescript
```

Branch protection – Gates de seguridad

```
main
|
├─ Require PR reviews (2 approvers)
├─ Require status checks to pass:
|   ├── ✓ npm audit (0 high/critical)
|   ├── ✓ gitleaks (0 secrets detected)
|   ├── ✓ CodeQL (0 errors)
|   └── ✓ Tests passing
├─ Require signed commits
└─ No force pushes
```

✓ **Nadie puede mergear a main si la seguridad falla.** Automático, sin excepciones.

Lab T19 – Crear GitHub Actions workflow

1. Crear `.github/workflows/security.yml` :

```
name: Security
on: [push]
jobs:
  audit:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci
      - run: npm audit --audit-level=high
      - uses: gitleaks/gitleaks-action@v2
```

2. Hacer push y verificar en la pestaña Actions

3. ¿Pasa o falla? ¿Por qué?

 **Resultado:** Cada push ejecuta auditoría + detección de secretos automáticamente.

T20: Pentesting con OWASP ZAP

Auditoría automatizada de vulnerabilidades

¿Qué es OWASP ZAP?

Zed Attack Proxy — herramienta DAST open source de OWASP:

- 🕷 **Spider:** rastrea todas las páginas/endpoints automáticamente
- ⚡ **Active Scan:** prueba ataques (SQLi, XSS, etc.) contra cada URL
- 📊 **Alerts:** genera informe con vulnerabilidades encontradas y severidad
- 🐳 **Docker:** se ejecuta sin instalación

```
# Ejecutar ZAP contra VulnApp:  
docker run -t zapproxy/zap-stable zap-baseline.py \  
-t https://vulnapp-owasp-01.azurewebsites.net  
  
# Baseline scan: pasivo (no ataca), ideal para CI/CD  
# Full scan: activo (prueba ataques), más lento
```

ZAP en Docker – Comandos principales

1. Baseline scan (rápido, pasivo, para CI/CD):

```
docker run -t zaproxy/zap-stable zap-baseline.py \  
  -t https://vulnapp-owasp-01.azurewebsites.net \  
  -r report.html
```

2. Full scan (activo, prueba ataques):

```
docker run -t zaproxy/zap-stable zap-full-scan.py \  
  -t https://vulnapp-owasp-01.azurewebsites.net \  
  -r full-report.html
```

3. API scan (para REST APIs):

```
docker run -t zaproxy/zap-stable zap-api-scan.py \  
  -t https://vulnapp-owasp-01.azurewebsites.net/api \  
  -f openapi \  
  -r api-report.html
```

Interpretando alertas de ZAP

```
WARN-NEW: 10021 X-Content-Type-Options Header Missing [Medium]
WARN-NEW: 10038 Content Security Policy (CSP) Header Not Set [Medium]
WARN-NEW: 40012 Cross Site Scripting (Reflected) [High]
WARN-NEW: 40018 SQL Injection [High]
WARN-NEW: 10035 Strict-Transport-Security Header Not Set [Low]
FAIL-NEW: 0      [FAIL - Total: 5, High: 2, Medium: 2, Low: 1]
```

Severidad	Significado	Acción
High	Explotable, impacto grave	 Fix inmediato
Medium	Riesgo moderado	 Fix en sprint actual
Low	Buena práctica ausente	 Planificar
Informational	Solo información	 Documentar

ZAP en CI/CD

```
# GitHub Actions - DAST con ZAP
dast:
  runs-on: ubuntu-latest
  needs: deploy # Solo tras desplegar
  steps:
    - name: OWASP ZAP Baseline Scan
      uses: zaproxy/action-baseline@v0.12.0
      with:
        target: 'https://vulnapp-staging.azurewebsites.net'
        fail_action: true # Falla si hay alertas High

    - name: Upload Report
      uses: actions/upload-artifact@v4
      with:
        name: zap-report
        path: report_html.html
```

Metodología de Pentesting – 5 fases

Fase	Actividad	Herramientas
1. Reconocimiento	Descubrir endpoints, tecnologías	nmap, Wappalyzer, robots.txt
2. Enumeración	Mapear la superficie de ataque	ZAP Spider, Burp, ffuf
3. Explotación	Intentar los ataques	SQLi, XSS, CMDi, SSRF
4. Post-explotación	Evaluar el impacto real	¿Qué datos se acceden?
5. Informe	Documentar hallazgos + fixes	Markdown/PDF con CVSS

Estructura de un informe de auditoría

```
# Informe de Auditoría de Seguridad – VulnApp

## Resumen Ejecutivo
- 3 Critical, 5 High, 8 Medium
- Riesgo general: ALTO

## Hallazgos

### [CRITICAL] SQL Injection en /api/products
- CVSS: 9.8 (Critical)
- Endpoint: GET /api/products?q=
- Payload: ' UNION SELECT username,password FROM users--
- Impacto: Acceso completo a la base de datos
- Fix: Usar prepared statements
- Referencia: CWE-89, OWASP A03:2021

### [HIGH] Stored XSS en /api/products
...
```


CVSS – Common Vulnerability Scoring System

CVSS Base Score = (Attack Vector + Complexity + Privileges + User Interaction)
× (Confidentiality + Integrity + Availability)

Score	Rating	Ejemplo del curso
9.0-10.0	Critical	SQLi sin auth (UNION SELECT passwords)
7.0-8.9	High	XSS almacenado que roba tokens
4.0-6.9	Medium	CSRF sin SameSite cookies
0.1-3.9	Low	Falta X-Content-Type-Options



Lab T20 — Escaneo ZAP

```
# Ejecutar baseline scan:  
docker run -t zapproxy/zap-stable zap-baseline.py \  
-t https://vulnapp-owasp-01.azurewebsites.net
```

Preguntas:

- # 1. ¿Cuántas alertas encuentra?*
- # 2. ¿Cuáles son High?*
- # 3. ¿Detecta la falta de CSP/HSTS?*
- # 4. ¿Detecta el SQLi automáticamente?*

 **Documenta:** Cada alumno escribirá 2 hallazgos del informe con formato: Título, CVSS, Payload, Impacto, Fix.

T21: OWASP Top 10 — Mapa Completo

Uniendo todo el curso

OWASP Top 10 (2021) – Las 10 categorías

#	Categoría	Nuestro tema
A01	Broken Access Control	T08 (IDOR), T09 (CSRF)
A02	Cryptographic Failures	T10 (TLS), T11 (Passwords)
A03	Injection	T04 (SQLi), T05 (XSS), T06 (CMDi)
A04	Insecure Design	T15 (SSRF), T08 (IDOR)
A05	Security Misconfiguration	T13 (Headers), T16 (DB)
A06	Vulnerable Components	T14 (Dependencies)
A07	Auth & Session Failures	T07 (JWT), T09 (Sessions)
A08	Data Integrity Failures	T14 (Supply chain), T19 (CI/CD)
A09	Logging & Monitoring	T19 (Pipeline), T20 (ZAP)
A10	SSRF	T15 (SSRF)

Lo que hemos aprendido – Stack completo

CI/CD (T19): npm audit + gitleaks + CodeQL	← Prevenir
Proxy (T18): Nginx + TLS + Rate Limit + WAF	← Proteger
App (T04-T09,T13): SQLi, XSS, CSRF, Headers	← Código seguro
Auth (T07,T11): JWT seguro + bcrypt/Argon2	← Identidad
Docker (T17): Multi-stage + USER + Trivy	← Contenedor
BD (T16): GRANT mínimo + red aislada + cifrado	← Datos
Secretos (T12): .env + Key Vault + rotación	← Configuración
Pentest (T20): ZAP + informe + CVSS	← Verificar

Recursos para seguir aprendiendo

Recurso	Tipo	Nivel
PortSwigger Web Security Academy	Labs interactivos	Principiante → Avanzado
Hack The Box	Máquinas vulnerables	Intermedio
TryHackMe	Rutas guiadas	Principiante
OWASP WebGoat	App vulnerable educativa	Principiante
PentesterLab	Ejercicios prácticos	Intermedio
CTFtime	Competiciones CTF	Todos

Certificaciones de seguridad — Roadmap

Principiante (0-1 año):

- └─ eJPT (INE) – Pentest Junior (~\$200)
- └─ CompTIA Security+ – Fundamentos (~\$400)
- └─ BSCP (PortSwigger) – Web Security (~\$100)

Intermedio (1-3 años):

- └─ eWPT (INE) – Web Pentest (~\$400)
- └─ OSCP (OffSec) – Pentest generalista (~\$1500)
- └─ CRTP – Active Directory (~\$250)

Avanzado (3+ años):

- └─ OSWE (OffSec) – Web avanzado (~\$1600)
- └─ OSED (OffSec) – Exploit Development
- └─ CPTS (HTB) – Pentest completo (~\$200)

Carreras en ciberseguridad

Rol	Foco	Salario EU (aprox.)
Security Engineer	Proteger infraestructura	45-70K€
Penetration Tester	Encontrar vulnerabilidades	50-80K€
AppSec Engineer	Código seguro + DevSecOps	55-85K€
Bug Bounty Hunter	Freelance, reportar vulns	Variable (0-500K\$)
SOC Analyst	Monitorización + respuesta	35-55K€
Cloud Security	AWS/Azure/GCP hardening	60-100K€

Lab T21 – Informe final

Ejercicio final del curso:

Cada alumno escribe un mini-informe de 3 vulnerabilidades de VulnApp:

```
## Hallazgo 1: [TÍTULO]
- OWASP: A0X
- CVSS: X.X
- Payload: ...
- Impacto: ...
- Fix recomendado: ...
```

 **Entregable:** Informe en Markdown con al menos 3 vulnerabilidades, clasificadas por CVSS, con fixes concretos.

Resumen del curso completo

Ataques practicados:

- SQL Injection (UNION, Blind)
- XSS (Stored, Reflected, DOM)
- Command Injection
- JWT manipulation
- IDOR
- CSRF
- SSRF (conceptual)

Defensas implementadas:

- Prepared statements
- sanitize-html + CSP
- execFile + allowlist
- JWT verify + strong key
- Ownership checks
- SameSite cookies
- Allowlist URLs + IP check
- Helmet, bcrypt, Key Vault
- Docker hardening, Nginx
- CI/CD pipeline con gates

Resumen del día

Tema	Problema	Solución
CI/CD	Seguridad al final (shift-right)	Pipeline: audit + gitleaks + CodeQL
Pentesting	No saber si la app es segura	OWASP ZAP + CVSS + informe
OWASP	No conocer las categorías	Top 10 mapeado a 21 temas prácticos

🏆 **¡Curso completado!** Tenéis los conocimientos para auditar, explotar y defender aplicaciones web. El siguiente paso es PRACTICAR.